

Budapesti Műszaki Szakképzési Centrum

Verebély László Technikum

506 131203 Szoftverfejlesztő és tesztelő

Vizsgaremek

EgészségÚt

/MedActivities/

Készítették:

1. Varga András Ernő
2. Maloschik Erik

Konzulens tanárok:

Somogyi Erika, Juhász Zoltán, Horváth Attila, Faludi Anita

2026. szeptember 15.

Tartalomjegyzék

1. Bevezetés	3
2. Fejlesztői dokumentáció	3
2.1. Alkalmazott fejlesztői eszközök	4
2.2. Alkalmazott technológiák	4
2.2.1. A felhasznált ismeretek	4
2.3. Futtatási környezet	4
2.4. Fejlesztői ütemterv	5
2.5. Fejlesztés menete	6
2.5.1. Piackutatás	6
2.5.2. Adatbázis megtervezése	7
2.5.3. Backend fejlesztés REST API	11
2.5.4. Felhasználói felület vázlatai	12
2.5.5. A webalkalmazás főbb algoritmusai	14
2.5.6. Az asztali alkalmazás főbb algoritmusai	21
2.5.7. Tesztelés	24
2.5.8. Későbbi továbbfejlesztési lehetőségek	28
3. Felhasználói dokumentáció	29
3.1. Webalkalmazás bemutatása	29
3.2. Az asztali alkalmazás bemutatása	32
4. Összegzés	36
5. Ábrajegyzék	37
6. Forrásjegyzék	38

1. Bevezetés

Az EgészségÚt / MedActivities egy többplatformos egészségügyi eseménykezelő és pácienskezelő rendszer. A szoftver célja, hogy a páciensek, orvosok és egészségügyi dolgozók egy központi rendszeren keresztül tudják kezelni az egészségügyi vizsgálatokhoz, kezelésekhöz és betegüteseményekhez kapcsolódó adatokat, dokumentumokat és megjegyzéseket.

A rendszer átlátható kommunikációt és adatmegosztást biztosít az ellátási folyamat szereplői között. Segíti annak követését, hogy egy páciensnek milyen vizsgálatra, kezelésre vagy további ellátási lépésre van szüksége.

2. Fejlesztői dokumentáció

Az EgészségÚt vagy MedActivities egy többplatformos egészségügyi eseménykezelő és pácienskezelő rendszer. A projekt célja, hogy a páciensek, orvosok és egészségügyi dolgozók egy központi rendszeren keresztül tudják kezelni az egészségügyi vizsgálatokhoz, kezelésekhöz és betegüteseményekhez kapcsolódó adatokat, dokumentumokat és megjegyzéseket. A projekt egy valós problémára nyújt megoldást, hogy egy közös alkalmazásban lehessen kezelni az egészségügyi eseményeket, amelyeken keresztül így egy kommunikációs útvonalat is kiépítünk a kezelőorvos, az egészségügyi szakdolgozó /„Practitioner”/ és a páciensek között. Ezáltal a webes applikáció segítséget nyújt az ellátási folyamat átláthatóbbá tételében. Lehetővé teszi, hogy az érintett szereplők nyomon kövessék, hogy egy páciensnek milyen vizsgálatra, kezelésre vagy további ellátási lépésre van szüksége.

Ezek az alkalmazások integrálhatóak a meglévő egészségügyi rendszerekbe, továbbá a következő funkciók használatára is alkalmas: a páciensek nyilvántartása, az egészségügyi események rögzítése és kezelése, a páciens és kezelő szakember kapcsolata, az időpontfoglalás, valamint a leletek feltöltése, rendszerezése is beépíthető továbbfejlesztési modul.

A backend ASP.NET Core REST API, az alkalmazási logika MediatR/CQRS mintát követ, az adatelérés Entity Framework Core-on keresztül történik, az MVP adatbázisa SQLite. A webes kliens React, az asztali kliens C# Windows Forms. A végleges architektúrában mindkét kliens ugyanazt az API-t használja.

A projekt kiindulópontja egy közismert social-media felület Activity-alapú alkalmazás részegysége, aminek megfelelő a vizuális felülete a széleskörű egyszerű használatra. Az Activity CRUD, MediatR, AutoMapper, EF Core és SQLite alapokra kerül rá a Patient, Practitioner, Patient–Activity kapcsolat, időpontfoglalás, Identity és validáció.

2.1. Alkalmazott fejlesztői eszközök

Kódszerkesztő: Visual Studio Code

Programfejlesztői környezet: Visual Studio

API tesztelő eszköz: Postman

Verziókezelés: Git és GitHub

Dokumentációkészítés: Microsoft Word

2.2. Alkalmazott technológiák

Backend: C# – ASP.NET Core Web API

Frontend web: React + TypeScript

Asztali alkalmazás: Windows Forms, C#

Adatbázis: Microsoft SQL Server

ORM: Entity Framework Core

Architektúra: Clean Architecture, CQRS, Mediator minta

2.2.1. A felhasznált ismeretek

A projekt során felhasznált ismeretek: objektumorientált programozás, adatbázis-kezelés, SQL, REST API fejlesztés, webfejlesztés, kliens-szerver architektúra, jogosultságkezelés, adatvalidáció, verziókezelés és dokumentációkészítés.

2.3. Futtatási környezet

A backend alkalmazás ASP.NET Core Web API környezetben fut. A rendszer a kliensek kéréseit REST API végpontokon keresztül fogadja, feldolgozza az üzleti logikát, majd az adatokat Microsoft SQL Server adatbázisban tárolja. Az adatbázis-elérést Entity Framework Core biztosítja.

A webes kliens React + TypeScript alapú alkalmazás, amely modern webböngészőben futtatható. A webes felület elsősorban a páciensek számára készül, és reszponzív kialakítása miatt asztali gépen, laptopon, tableten és mobiltelefonon is használható.

Az egészségügyi dolgozók számára készült asztali kliens Windows Forms technológiával készül, ezért Windows operációs rendszeren futtatható. Ez az alkalmazás adminisztrációs felületként működik, és ugyanazt a REST API-t használja, mint a webes kliens.

A rendszer futtatásához szükséges főbb környezeti elemek:

Komponens	Futtatási környezet
Backend API	.NET futtatókörnyezet, ASP.NET Core Web API.
Webes kliens	Modern böngésző, például Chrome, Edge vagy Firefox.
Asztali kliens	Windows operációs rendszer, Windows Forms alkalmazásként.
Adatbázis	Microsoft SQL Server

A fejlesztői gépen a backend és az adatbázis lokálisan is futtatható. Éles vagy bemutató környezetben a backend szerver, az SQL Server adatbázis és a kliensek külön komponensként működhetnek, de ugyanazon API-n keresztül kommunikálnak egymással.

2.4. Fejlesztői ütemterv

Határidő	Feladat
2026. május	Vizsgaremetéma kiválasztása, csoportalakítás.
2026. június	Projektleírás, követelményelemzés és felhasználói felület vázlatainak elkészítése.
2026. július	Adatbázis-terv, entitások és kapcsolatok meghatározása.
2026. augusztus	Backend REST API, autentikáció és jogosultságkezelés kialakítása.
2026. szeptember	Webalkalmazás és Windows Forms asztali alkalmazás fejlesztése.
2026. szeptember	Tesztelés, hibajavítás és dokumentáció véglegesítése.

Követelménymátrix

ID	Követelmény	Prioritás
K1	Felhasználói regisztráció és bejelentkezés	Magas
K2	Szerepköralapú jogosultságkezelés	Magas
K3	Páciensadatok kezelése	Magas
K4	Egészségügyi események rögzítése	Magas
K5	Dokumentumok csatolása eseményekhez	Magas

ID	Követelmény	Prioritás
K6	Megjegyzések kezelése	Közepes
K7	Betegút-lépések követése	Magas
K8	Értesítések kezelése	Közepes
K9	Naplózás és alapvető auditálás	Közepes
K10	Időpontfoglalás	Alacsony

2.5. Fejlesztés menete

2.5.1. Piackutatás

A fejlesztés elején áttekintettük az egészségügyi időpontfoglaló, pácienskezelő és dokumentumkezelő rendszerek jellemző funkcióit. A vizsgálat alapján fontos igényként jelent meg az átlátható betegút-követés, az eseményekhez kapcsolható dokumentumkezelés és az orvos-páciens kommunikáció támogatása.

A projekt egészségügyi ellátási események kezelésére épül, az időpontfoglalás kiegészítő funkcióként jelenik meg, amit további fejlesztésnél be lehet építeni igény esetén vagy integrálni az időpontfoglaló rendszerbe, ami most működik Magyarországon. Ezért a rendszer központi eleme az egészségügyi esemény, amelyhez vizsgálati adatok, kezelési tervek, dokumentumok, megjegyzések és további ellátási lépések kapcsolhatók.

2.5.2. Adatbázis megtervezése

Az EgészségÚt / MedActivities rendszer adatbázisa relációs adatmodellre épül. A fejlesztési/MVP változat SQLite adatbázist használ, mert egyszerűen futtatható helyi környezetben. A végleges rendszerhez Microsoft SQL Server használata indokolt, mert jól illeszkedik az ASP.NET Core és az Entity Framework Core felépítéshez.

A rendszer központi entitása az egészségügyi esemény. Egy egészségügyi esemény lehet például vizsgálat, kezelés, konzultáció, kontroll vagy beutalás. Ehhez kapcsolódhatnak dokumentumok, megjegyzések, valamint további betegút-lépések.

Táblanév	Fontosabb mezők	Leírás
AspNetUsers, AspNetRoles	Id, UserName, Email, PasswordHash, szerepkörkapcsolatok	Identity-alapú felhasználók és jogosultságok.
Patients	Id, UserId, Name, TajNumber, BirthDate, BirthPlace, Email, Phone, Address	Páciensek alapadatai; a TAJ egyedi, szövegesen tárolt azonosító.
Practitioners	Id, UserId, Name, TajNumber, Specialty, City, Venue	Kezelőorvosi profilok és rendelési adatok.
Activities	Id, Title, Date, Description, Category, Status, City, Venue, audit mezők	Vizsgálatok, kezelések, kontrollok és betegütesemények.
PatientActivities	PatientId, ActivityId	Kapcsolótábla páciensek és események között.
ActivityPractitioners	PractitionerId, ActivityId	Kapcsolótábla kezelőorvosok és események között.
Appointments	Id, PatientId, PractitionerId, ActivityId, BookingDate, StartTime, EndTime, Status	Időpontfoglalások; ellenőrzött dátum/idősáv és ütközésvédelem.
PatientDocuments	Id, PatientId, Title, FileName, ContentType, Content, UploadedByUserId	Pácienshez kapcsolt dokumentumok, SQL Serveren varbinary(max) tartalommal.

Validációs terület	Szabály
E-mail cím	Nem lehet üres, és egyedinek kell lennie.
Jelszó	Csak titkosított hash formában tárolható.
Szerepkör	Csak előre meghatározott szerepkör használható.
TAJ-szám	Ha meg van adva, egyedinek kell lennie.
Időpont	A befejezés időpontja későbbi legyen a kezdésnél.
Dokumentum	A fájlnev, fájlútvonal, fájl típus és feltöltő felhasználó kötelező.
Megjegyzés	Üres megjegyzés nem menthető.

A legfontosabb kapcsolatok:

- Egy felhasználóhoz szerepkörtől függően páciens-, kezelőorvosi, adminisztrátori vagy felvételi irodai profil kapcsolódhat.
- Egy pácienshez több egészségügyi esemény, időpont, dokumentum és megjegyzés tartozhat.
- Egy eseményhez több páciens és több kezelőorvos is rendelhető a kapcsolótáblákon keresztül.
- Egy időpont egy pácienshez, egy kezelőorvoshoz és egy eseményhez kapcsolódik.
- A PatientPractitionerAccesses tábla határozza meg, mely kezelőorvos mely páciens adataihoz férhet hozzá.

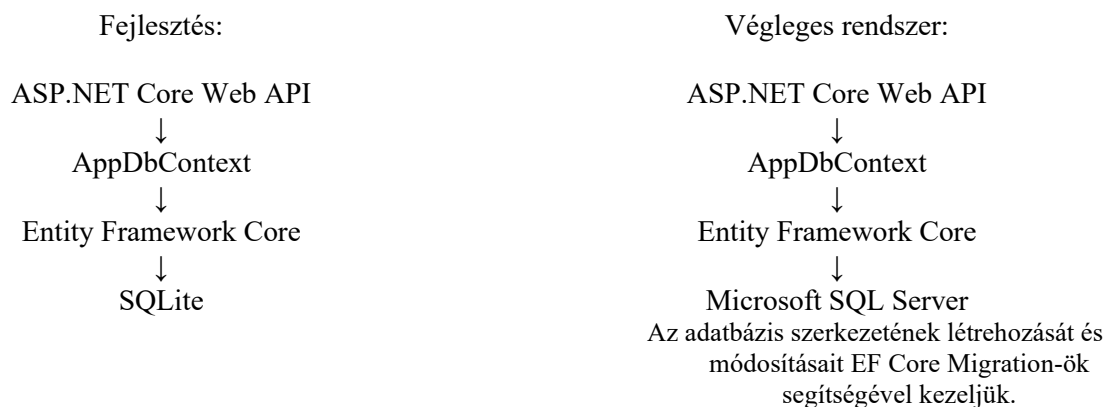
Validációk és integritási szabályok:

- A páciensek és kezelőorvosok TAJ-száma egyedi indexen keresztül védett.
- Az időpontoknál check constraint ellenőrzi, hogy a kezdési és zárási idő a foglalási dátumhoz és egész órás rendelési időszávhhoz illeszkedjen.
- Egy aktív páciens ugyanarra a napra, illetve egy kezelőorvos ugyanarra az időszávr nem kaphat ütköző foglalatást.
- A dokumentumok címe, fájlneve, típusa és mérete alkalmazásoldali validáción is átesik.
- A törlési archívum triggerrel készül, így fontos üzleti rekord törlésekor megmarad a törlés előtti állapot JSON másolata.

Az EgészségÚt / MedActivities fejlesztés kezdeti szakaszában SQLite adatbázist alkalmaztunk. Ennek előnye, hogy egyszerűen konfigurálható, nem igényel külön adatbázis-szervert, ezért megfelelő a rendszer fejlesztéséhez és teszteléséhez.

Az adatbázis elérését a backendben az Entity Framework Core (EF Core) biztosítja. Az alkalmazás és az adatbázis közötti kapcsolat központi eleme az AppDbContext, amelyen keresztül az alkalmazás az entitások lekérdezését, létrehozását, módosítását és törlését végzi.

A projekt végleges változatában Microsoft SQL Server használatát tervezzük. Az EF Core alkalmazása megkönnyíti az SQLite-ről SQL Serverre történő átállást, mivel az alkalmazás adatkezelési logikájának jelentős része adatbázis-kezelőtől független maradhat.

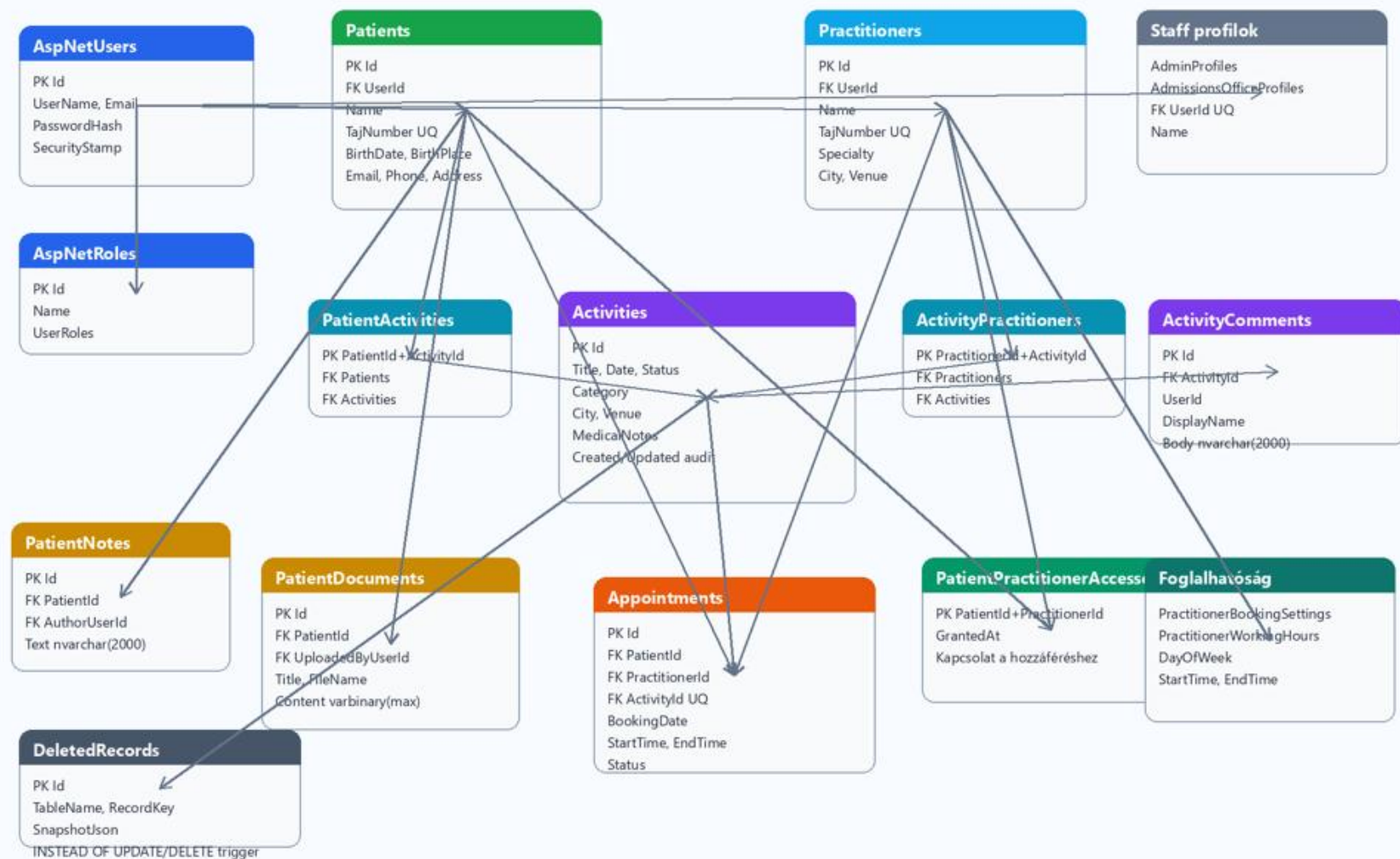


Az ER diagram az adatbázis fő entitásait és azok kapcsolatait szemlélteti. A diagramon látható, hogy a felhasználókhöz páciens- vagy dolgozói profil kapcsolódik, az egészségügyi események pedig a páciensekhez, időpontokhoz, dokumentumokhoz, megjegyzésekhez és betegút-lépésekhez kötődnek.

Az 1. ábra, vagyis az ER diagram alapján a rendszer adatmodellje alkalmas arra, hogy a páciensekhez kapcsolódó egészségügyi eseményeket, dokumentumokat és további ellátási lépéseket egységesen kezelje. A modell támogatja a többplatformos működést, mert a webes felület és az asztali alkalmazás ugyanazt a központi adatbázist használja a REST API-n keresztül.

EgészségÚt / MedActivities - egyszerűsített SQL Server ER diagram

A végleges beadási profil Microsoft SQL Server adatbázist használ EF Core migrációkkal.



1. ábra: Az EgészségÚt / MedActivities adatbázisának egyszerűsített ER diagramja

2.5.3. Backend fejlesztés REST API

A backend ASP.NET Core Web API technológiával készült. A webalkalmazás és a Windows Forms pácienskezelő kliens ugyanazt a HTTP/JSON alapú API-t használja, a tényleges adatkezelés pedig a Microsoft SQL Server adatbázison történik EF Core-on keresztül.

- Backend technológia: C# és ASP.NET Core Web API.
- Adatbázis: Microsoft SQL Server, EF Core migrációkkal.
- Kommunikáció: REST API JSON kérésekkel és válaszokkal.
- Hitelesítés: ASP.NET Identity és bearer token alapú munkamenet.
- Valós idejű kommunikáció: SignalR alapú eseménychat.

Metódus	URL	Leírás	Request példa
GET	/api/health	API és adatbáziskapcsolat ellenőrzése	-
POST	/api/session/register	Páciens vagy kezelőorvos regisztrációja	{ "userName": "paciens1", "role": "Patient" }
POST	/api/session/login	Bejelentkezés bearer tokennel	{ "userName": "admin@example.undefiend", "password": "****" }
GET	/api/session/me	Aktuális felhasználó és szerepkörök lekérdezése	-
GET	/api/activities	Látható egészségügyi események listázása	-
GET	/api/activities/page	Lapozott és szűrt eseménylista	?page=1&category=Vizsgálat
GET	/api/activities/{id}	Esemény részletei	-
POST	/api/activities	Új esemény létrehozása	{ "title": "Kontroll", "patientId": "..." }
PUT	/api/activities	Esemény módosítása	{ "id": "...", "status": "Scheduled" }
DELETE	/api/activities/{id}	Esemény törlése	-
GET	/api/activities/assignment-options	Páciens/kezelő hozzárendelési opciók	-
GET	/api/patients	Páciensek listája jogosultság szerint	-
POST	/api/patients	Új páciens létrehozása	{ "name": "Teszt Páciens", "tajNumber": "123456789" }

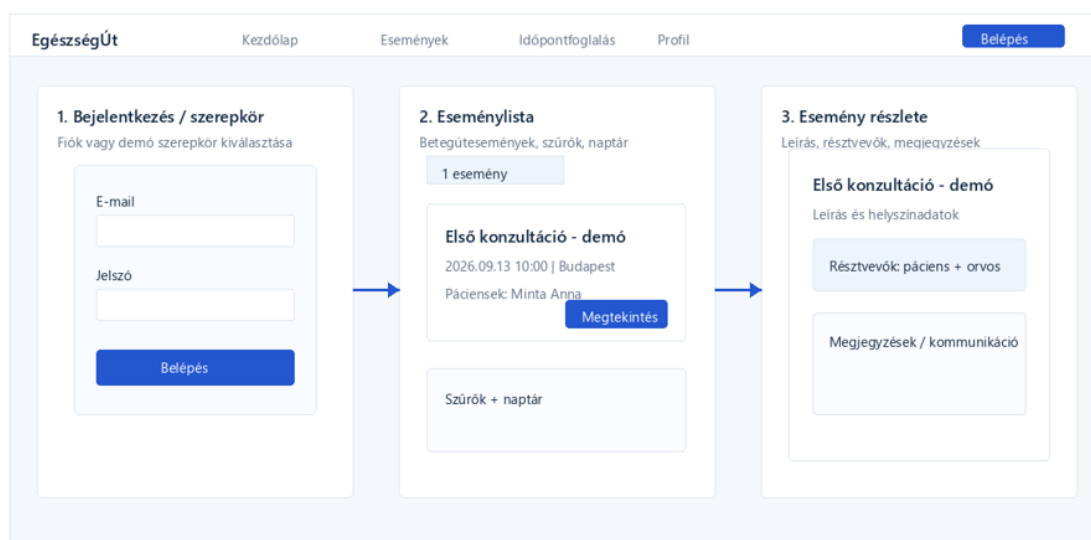
2.5.4. Felhasználói felület vázlatai

A rendszer két fő felülettel rendelkezik: egy reszponzív webes felülettel a páciensek számára, valamint egy Windows Forms alapú asztali alkalmazással az egészségügyi dolgozók adminisztrációs feladataihoz.

Működési logika

A felhasználó regisztrál vagy bejelentkezik. A rendszer szerepkör alapján jogosultságot rendel hozzá. A páciens időpontot foglalhat, dokumentumot tölthet fel, illetve megtekintheti saját eseményeit. Az egészségügyi dolgozó eseményt hozhat létre, megjegyzést írhat, dokumentumot csatolhat, és betegút-lépést rögzíthet. A rendszer értesítést küldhet a fontos változásokról.

Webalkalmazás drótváz - páciens betegút és eseménykezelés



2. ábra: A webalkalmazás fő felhasználói folyamatának drótváza

A felhasználó e-mail címmel és jelszóval tud belépni a rendszerbe. A felhasználó (páciens) áttekintheti saját egészségügyi eseményeit, időpontjait, dokumentumait és betegút-lépéseit.

Felhasználói szerepkörök

Admin / ADMIN

Jogosultság -> Fő feladat

Teljes adminisztráció; páciensek, kezelők, események, szerepkörök és foglalkozás kezelése.

Felvételi iroda / ADMIN

Páciensek és kezelők operatív kezelése, kapcsolatok és időpontok adminisztrációja.

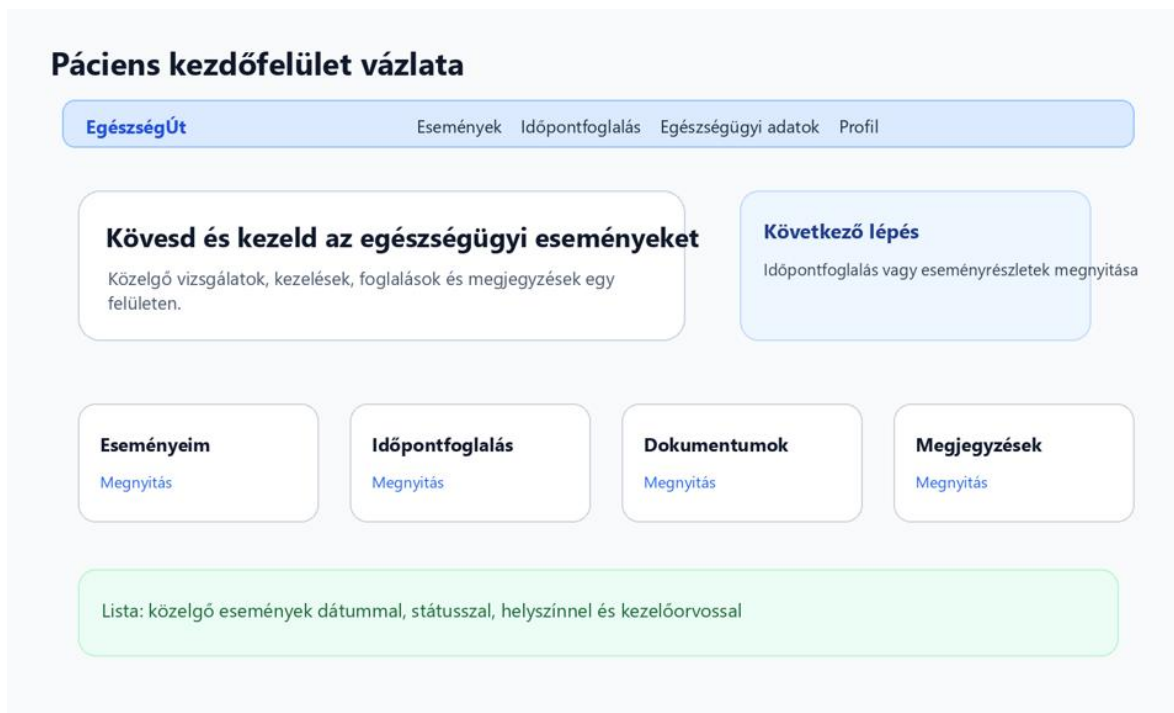
Páciens / USER

Saját adatok és események megtekintése, saját időpont foglalása.

Practitioner / USER

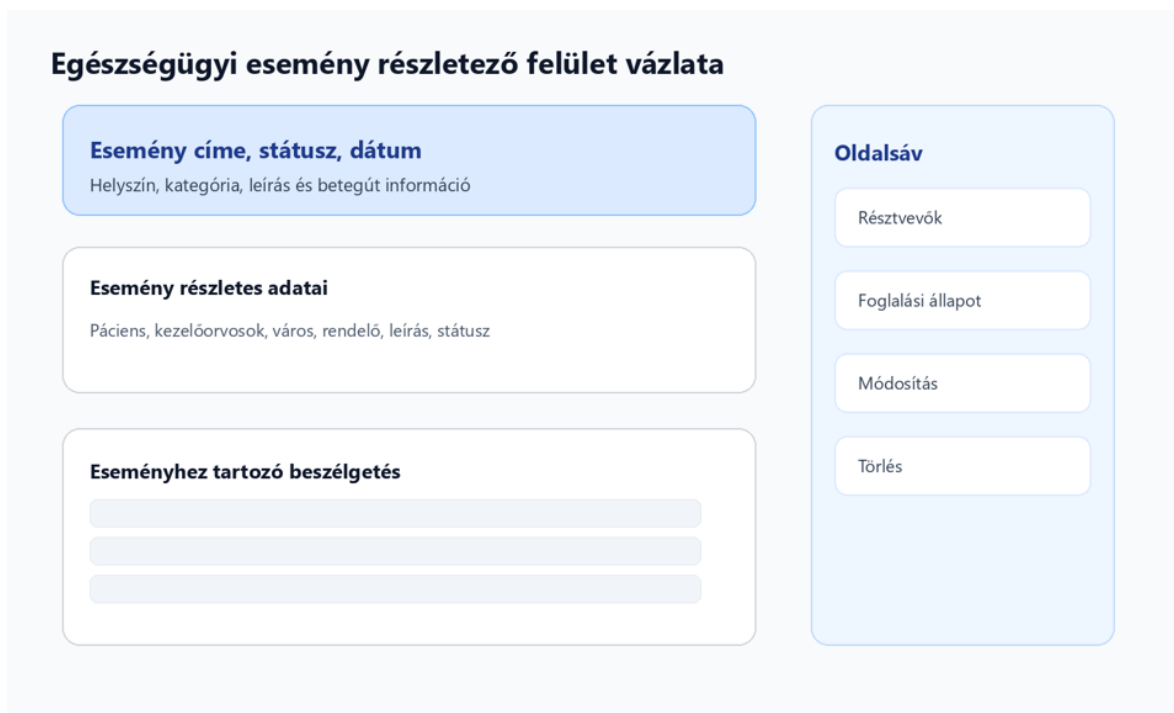
Kezelőorvos / Egészségügyi szakdolgozó

Saját események, hozzárendelt páciensek és saját munkarend kezelése jogosultság szerint.

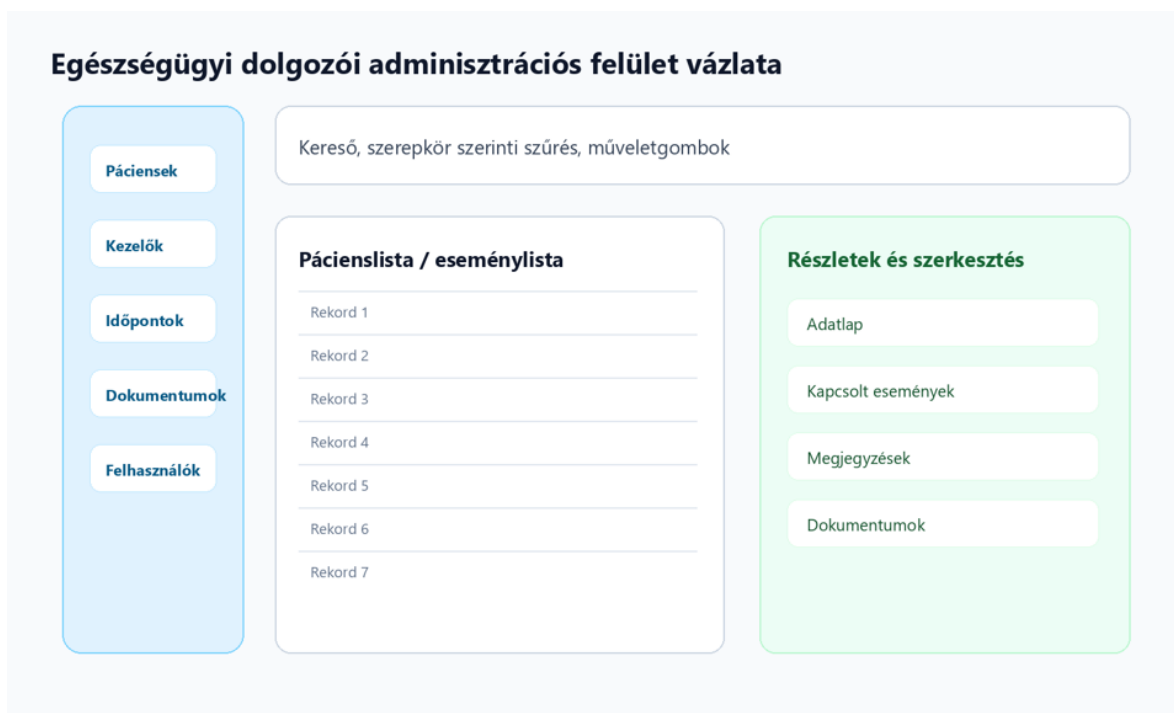


3. ábra: Páciens kezdőfelület vázlata

Egy konkrét vizsgálathoz, kezeléshez vagy betegüteseményhez kapcsolódó adatok jelennek meg.



4. ábra: Egészségügyi esemény részletező felület vázlata



5. ábra: Egészségügyi dolgozói adminisztrációs felület vázlata

A webes és az asztali kliens ugyanazt a központi REST API-t használja. A Windows Forms alkalmazás az egészségügyi dolgozók adminisztrációs munkáját támogatja.

2.5.5. A webalkalmazás főbb algoritmusai

A webalkalmazás főbb algoritmusai a backend REST API és a React kliens együttműködését mutatják be. A backendben a vezérlők, a MediatR Query/Command osztályok, az EF Core adatbázis-kapcsolat és a jogosultsági szolgáltatások működnek együtt. A frontend oldalon az Axios agent, a React Query cache, a Zod validáció és az űrlapkomponensek biztosítják a felhasználói folyamatot.

Backend példák

Forrásfájl	API/Controllers/ActivitiesController.cs
Szerep	Az ActivitiesController fogadja a HTTP kéréseket, majd a Mediatoron keresztül továbbítja a műveletet az Application rétegnek.

REST API végpontok vezérlése
C#

```

1 [HttpGet, AllowAnonymous]
2 public async Task<IActionResult> GetActivities(CancellationTok
3     string? patientId = null, string? practitionerId = null)
4 {
5     if (!Authenticated) return Unauthorized();
6     return Ok(await mediator.Send(new GetActivityList.Query(
7         access.UserId,
8         User.FindAll(ClaimTypes.Role).Select(c => c.Value).ToArray(),
9         patientId, practitionerId), ct));
10 }
11
12 [HttpPost, Authorize(Roles = "Admin,AdmissionsOffice,Practitioner")]
13 public async Task<IActionResult> CreateActivity(ActivityWriteInput input, CancellationTok
14     ct)
15 {
16     if (input.Date == default)
17         return BadRequest(new { message = "A dátum kötelező." });
18     // Validáció, jogosultság, kapcsolatok, mentés...
19 }

```

6. ábra: REST API végpontok vezérlése kódrészlete

Forrásfájl	Application/Activities/Queries/GetActivityList.cs
Szerep	A Query és Handler páros az olvasási műveletet különíti el, és aszinkron módon kéri le az Activity rekordokat.

Események listázása CQRS Queryvel
C#

```

1 public record Query(string UserId, string[] Roles,
2     string? PatientId = null,
3     string? PractitionerId = null) : IRequest<List<ActivityDto>>;
4
5 public class Handler(AppDbContext context)
6     : IRequestHandler<Query, List<ActivityDto>>
7 {
8     public async Task<List<ActivityDto>> Handle(Query request, CancellationTok
9         => await ActivityVisibility.For(context, request.UserId, request.Roles)
10             .Where(a => request.PatientId == null ||
11                 a.PatientActivities.Any(p => p.PatientId == request.PatientId))
12             .Where(a => request.PractitionerId == null ||
13                 a.ActivityPractitioners.Any(p => p.PractitionerId ==
14                     request.PractitionerId))
15             .AsNoTracking()
16             .OrderByDescending(a => a.Date)
17             .Select(ActivityDto.Projection)
18             .ToListAsync(ct);
19 }

```

7. ábra: Események listázása CQRS Queryvel kódrészlete

Forrásfájl	Application/Activities/Commands/CreateActivity.cs
Szerep	A Command az új egészségügyi esemény mentését indítja, a Handler pedig az AppDbContexten keresztül véglegesíti.

Új esemény rögzítése Commanddal

C#

```

1 public class CreateActivity
2 {
3     public class Command : IRequest<string>
4     {
5         public required Activity Activity { get; set; }
6     }
7
8     public class Handler(AppDbContext context) : IRequestHandler<Command, string>
9     {
10        public async Task<string> Handle(Command request, CancellationToken ct)
11        {
12            context.Activities.Add(request.Activity);
13            await context.SaveChangesAsync(ct);
14            return request.Activity.Id;
15        }
16    }
17 }

```

8. ábra: Új esemény rögzítése Commanddal kódrészlete

Forrásfájl	Application/Activities/Commands/EditActivity.cs
Szerep	A módosító algoritmus megkeresi a meglévő rekordot, majd AutoMapper-rel átvezeti az új adatokat.

Esemény módosítása AutoMapper-rel

Application/Activities/Commands/EditActivity.cs

```

1 using Domain;
2 using MediatR;
3 using Persistence;
4 using AutoMapper;
5
6 namespace Application.Activities.Commands;
7
8 public class EditActivity
9 {
10    public class Command : IRequest
11    {
12        public required Activity Activity { get; set; }
13        ...
14
15        mapper.Map(request.Activity, activity);
16
17        // activity.Title = request.Activity.Title; // -> Automappert-> ezt ahelyett
18        használjuk a mapper-t, ami automatikusan átviszi az értékeket a request.Activity-ből az
19        activity-be
20
21        await context.SaveChangesAsync(cancellationTokens);
22    }
23 }
24 }

```

Esemény módosítása a végleges kódban

C#

```

1 var target = await context.Activities.FindAsync([request.Activity.Id], cancellationToken)
2 ?? throw new Exception("Activity not found");
3
4 var source = request.Activity;
5 // Kapcsolatok, orvosi megjegyzések és státusz megmaradnak.
6 target.Title = source.Title;
7 target.Date = source.Date;
8 target.Description = source.Description;
9 target.Category = source.Category;
10 target.City = source.City;
11 target.Venue = source.Venue;
12 target.Latitude = source.Latitude;
13 target.Longitude = source.Longitude;
14 target.UpdatedAt = DateTime.UtcNow;
15
16 await context.SaveChangesAsync(cancellationTokens);

```

9. ábra: Esemény módosítása több módszerrel kódrészlete

Forrásfájl	API/Program.cs
Szerep	Az alkalmazásindítás regisztrálja az adatbázis-kapcsolatot, a CORS szabályokat, a MediatR és AutoMapper szolgáltatásokat.



10. ábra: Adatbázis és szolgáltatások regisztrálása kódrészlete

Frontend példák

Forrásfájl	client/src/lib/api/agent.ts
Szerep	Az Axios agent egységes API belépési pontot és központi HTTP hibakezelést biztosít a React frontend számára.



11. ábra: Központi API kliens és hibakezelés kódrészlete

Forrásfájl	client/src/lib/hooks/useActivities.ts
Szerep	A hook az eseménylista, részletes esemény, létrehozás, módosítás és törlés műveleteit fogja össze.



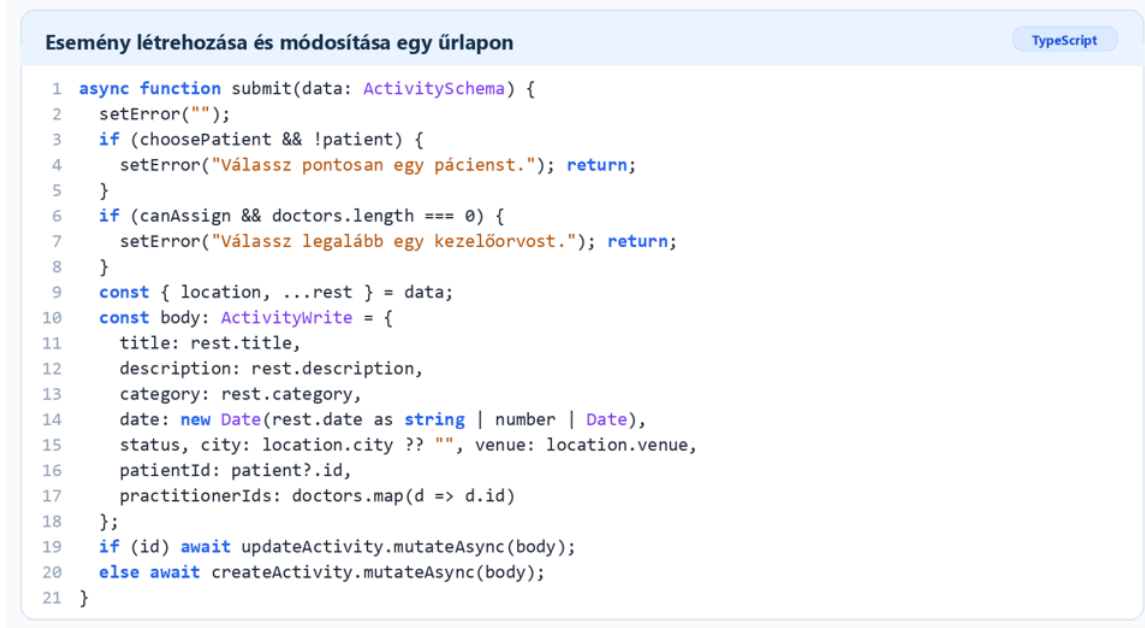
12. ábra: Adatlekérés és cache React Queryvel kódrészlete

Forrásfájl	client/src/lib/schemas/activitySchema.ts
Szerep	A Zod séma kliensoldalon ellenőrzi a kötelező mezőket, a dátumot és a helyszínhez tartozó adatokat.



13. ábra: Űrlapvalidáció Zoddal kódrészlete

Forrásfájl	client/src/features/activities/form/ActivityForm.tsx
Szerep	Az űrlap új esemény létrehozására és meglévő esemény szerkesztésére is használható.



14. ábra: Esemény létrehozása és módosítása egy űrlapon kódrészlete

Forrásfájl	client/src/features/activities/details/ActivityDetailsPage.tsx és ActivityDetailsChat.tsx
Szerep	A részletező oldal URL alapján tölti be az eseményt, a megjegyzésmező pedig az orvos-páciens kommunikáció alapját adja.

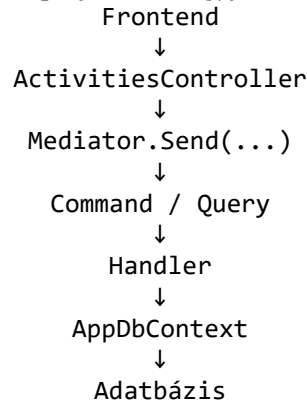


15. ábra: Részletező nézet és megjegyzésfelület kódrészlete

MediatR hogyan működik

A MediatR közvetítő réteggént választja el a controller szintet az alkalmazási logikától. A Controller egy Query vagy Command objektumot küld, a MediatR pedig megkeresi a megfelelő Handler osztályt. Így elkülönül a HTTP réteg, az üzleti logika és az adatbázis-kezelés.

A folyamat a projektben nagyjából így épül fel:



MediatR működése a MedActivities backendben



16. ábra: A MediatR közvetítő szerepe a frontend és az adatbázis között

```

1 return Ok(await mediator.Send(new GetActivityList.Query(
2     access.UserId,
3     User.FindAll(System.Security.Claims.ClaimTypes.Role)
4         .Select(c => c.Value)
5         .ToArray(),
6     patientId,
7     practitionerId,
8     ct));

```

17. ábra: *GetActivityList.Query* továbbítása MediatR segítségével

A MediatR megkeresi azt a Handler osztályt, amelyik ezt a CreateActivity.Command típust kezeli, majd lefuttatja. Ez azért előnyös, mert különválik a HTTP/API réteg, az üzleti logika és az adatbázis-kezelés. Ez a felépítés jól illik a CQRS mintához.

2.5.6. Az asztali alkalmazás főbb algoritmusai

Az asztali alkalmazás főbb algoritmusai az egészségügyi dolgozók munkafolyamataihoz kapcsolódnak. A legfontosabb folyamatok a pácienskeresés, az eseményrögzítés, a dokumentumcsatolás és a betegút-lépés állapotának módosítása. A végleges szerkezetben a közös ASP.NET Core API-hoz kapcsolódik, és azon keresztül ellenőrzi a SQL Server adatbázis elérhetőségét. A Windows Forms kliens egészségügyi dolgozói munkafolyamatokra készült: pácienskeresés, páciensadatok kezelése, események megjelenítése, foglalások kezelése, valamint dokumentumok és megjegyzések kezelése.

- A kliens induláskor ellenőrzi az API health végpontját, és csak SQL Server adatbázissal működő API-t fogad el.
- A pácienslista betöltése után a keresés kliensoldali szűréssel segíti a gyors kiválasztást.
- A páciens- és Activity-űrlapok a mentés előtt formai validációt végeznek.
- A rögzítés és módosítás HTTP POST/PUT kérésekkel történik, ezért a jogosultságellenőrzés továbbra is a backendben érvényesül.

```

1 private void SaveButton_Click(object? sender, EventArgs e)
2 {
3     if (string.IsNullOrWhiteSpace(nameBox.Text) ||
4         tajBox.Text.Length != 9 ||
5         tajBox.Text.Any(c => c is < '0' or > '9') ||
6         birthPicker.Value.Date > DateTime.Today)
7     {
8         MessageBox.Show(this,
9             "Név, pontosan 9 számjegyű TAJ és érvényes születési dátum szükséges.");
10        return;
11    }
12    if (!string.IsNullOrWhiteSpace(emailBox.Text) &&
13        !new EmailAddressAttribute().IsValid(emailBox.Text.Trim()))
14    {
15        MessageBox.Show(this, "Az e-mail-cím nem érvényes.");
16        return;
17    }
18    DialogResult = DialogResult.OK;
19 }

```

18. ábra: Páciensadatok validálása és mentése kódrészlete

Kiválasztott páciens Activity rekordjainak betöltése

C#

```

1 private async Task LoadPatientData()
2 {
3     var patient = SelectedPatient();
4     activityGrid.DataSource = bookingGrid.DataSource = null;
5     loadedPatientId = null;
6     if (patient is null) return;
7
8     selectedLabel.Text = patient.Name + " · TAJ: " + patient.TajNumber;
9     var activities = await api!.Get<List<ActivityItem>>("activities");
10    activityGrid.DataSource = activities
11        .Where(a => a.Patients.Any(p => p.Id == patient.Id))
12        .Select(a => new {
13            Azonosító = a.Id, Cím = a.Title,
14            Dátum = a.Date, Állapot = a.Status, Helyszín = a.Venue
15        }).ToList();
16    activityGrid.Columns["Azonosító"].Visible = false;
17 }

```

19. ábra: Kiválasztott páciens Activity rekordjainak betöltése kódrészlete

Activity adatok validálása és mentése

C#

```

1 public object Input => new {
2     id,
3     title = titleBox.Text.Trim(),
4     date = datePicker.Value,
5     description = descriptionBox.Text.Trim(),
6     category = categoryBox.Text.Trim(),
7     city = cityBox.Text.Trim(),
8     venue = venueBox.Text.Trim(),
9     status = statusBox.SelectedItem?.ToString(),
10    patientId = (patientBox.SelectedItem as Person)?.Id,
11    practitionerIds = doctorList.CheckedItems
12        .Cast<Person>().Select(p => p.Id).ToArray()
13 };
14
15 private void SaveButton_Click(object? sender, EventArgs e)
16 {
17     if (new[] { titleBox.Text, descriptionBox.Text, categoryBox.Text, venueBox.Text }
18         .Any(string.IsNullOrEmpty) ||
19         patientBox.SelectedItem is null || doctorList.CheckedItems.Count == 0)
20         return;
21     DialogResult = DialogResult.OK;
22 }

```

20. ábra: Activity adatok validálása és mentése kódrészlete

Pácienskeresés kliensoldali szűréssel

C#

```
1 private void FilterPatients()
2 {
3     var term = searchBox.Text.Trim();
4     patientGrid.DataSource = patients
5         .Where(p => p.Name.Contains(term, StringComparison.CurrentCultureIgnoreCase) ||
6             p.TajNumber.Contains(term))
7         .Select(p => new {
8             Azonosító = p.Id,
9             Név = p.Name,
10            TAJ = p.TajNumber,
11            Születés = p.BirthDate,
12            Email = p.Email,
13            Telefon = p.Phone
14        }).ToList();
15     patientGrid.Columns["Azonosító"].Visible = false;
16 }
```

21. ábra: Pácienskeresés SQL LIKE feltétellel kódrészlete

Activity rögzítése API-n keresztül

C#

```
1 private async Task EditActivity(bool edit) => await Run(async () =>
2 {
3     var patient = SelectedPatient(); if (patient is null) return;
4     var id = activityGrid.CurrentRow?.Cells["Azonosító"].Value?.ToString();
5     if (edit && id is null) return;
6
7     var options = await api!.Get<AssignmentOptions>("activities/assignment-options");
8     var item = edit ? await api.Get<ActivityItem>("activities/" + id) : null;
9
10    using var form = new ActivityEditForm();
11    form.Configure(options, patient.Id, item);
12    if (form.ShowDialog(this) != DialogResult.OK) return;
13
14    if (edit) await api.Put("activities", form.Input);
15    else await api.Post("activities", form.Input);
16    await LoadPatientData();
17 });
```

22. ábra: Activity rögzítése adatbázisba kódrészlete

2.5.7. Tesztelés

A tesztelés célja annak igazolása, hogy a rendszer normál és hibás bemenetek esetén is megfelelően működik. A backend végpontokat Postman segítségével CRUD szemléletben áteszteltük, a weboldal HTML helyességét pedig validátorral ellenőriztük, a felületeket pedig manuális funkcionális tesztekkel ellenőriztük le:

Teszt eset	Bemenet	Elvárt eredmény
Érvényes bejelentkezés	Helyes e-mail és jelszó	Sikeres belépés, token visszaadása.
Üres bejelentkezési mezők	Üres e-mail vagy jelszó	Validációs hibaüzenet.
Új egészségügyi esemény	Érvényes páciens és eseménytípus	Az esemény létrejön.
Jogosultsági teszt	Páciens más páciens adatát kéri	Hozzáférés megtagadva.

A webes felületnél ellenőriztük, hogy a fő oldalak reszponzívan jelennek meg, az űrlapok hibás bemenetnél validációs üzenetet adnak, és a böngészőben megnyitott HTML szerkezet validátorral nem tartalmaz kritikus szerkezeti hibát. A validálás különösen a kezdőlap, az eseménylista, az eseményrészletező és a bejelentkezési folyamat szempontjából fontos.

ID	Végpont	Tesztelt művelet	Elvárt eredmény	Eredmény
BE-01	POST /api/session/login	Bejelentkezés helyes és hibás adatokkal	Sikeres token vagy 401-es hiba	Megfelelt
BE-02	GET /api/activities	Eseménylista lekérdezése bejelentkezett felhasználóval	Szerepkör szerinti lista	Megfelelt
BE-03	POST /api/activities	Új egészségügyi esemény létrehozása	201 Created és új rekord	Megfelelt
BE-04	GET /api/activities/{id}	Esemény részleteinek lekérése	Részletes JSON válasz	Megfelelt
BE-05	PUT /api/activities	Esemény adatainak módosítása	204 NoContent és frissült adat	Megfelelt
BE-06	DELETE /api/activities/{id}	Teszt esemény törlése	204 NoContent vagy jogosultsági hiba	Megfelelt
BE-07	GET/POST /api/patients/{id}/records/notes	Megjegyzés listázása és rögzítése	Új megjegyzés megjelenik	Megfelelt
BE-08	GET /api/health	Adatbáziskapcsolat ellenőrzése	status="ok", database="SqlServer"	Megfelelt

Az EgészségÚt (MedActivities) rendszer szerepkör-alapú hozzáférési követelményeinek ellenőrzése. A teszteléshez egy privát, interaktív jogosultságpróba készült, amely minta-páciensekkel és mintakezelőkkel modellezi az Admin, Patient, Practitioner és Felvételi iroda jogosultságait.

i Adatvédelmi kialakítás: a prototípus kizárólag demonstrációs mintaadatokat használ, valós egészségügyi vagy személyes adatot nem tartalmaz, és tulajdonosi hozzáférésű privát Site-ként működik.

A jogosultsági teszt célja annak igazolása, hogy a rendszerben megfogalmazott hozzáférési szabályok következetesen érvényesülnek a három védett erőforrástípuson: a páciens adatlapján, az egészségügyi

eseményeken és a kezelő–páciens kapcsolatokon. Minden erőforrásnál külön vizsgáltuk a megtekintési és a szerkesztési jogosultságot.

Az Admin minden páciens, eseményt és kapcsolatot megtekinthet és szerkeszthet.

A Patient kizárólag a saját adatait és eseményeit éri el, valamint a saját kezelői hozzáféréseit szabályozhatja.

A Practitioner csak az aktív kapcsolattal hozzá rendelt pácienseket és azok eseményeit láthatja; az eseményszerkesztés külön engedélyhez kötött.

A Felvételi iroda létrehozhatja és bonthatja a kezelő–páciens kapcsolatokat, de az egészségügyi események tartalmát nem láthatja és nem szerkesztheti.



23. ábra – A jogosultsági modell szereplői és védett erőforrásai

A tesztkörnyezet egy böngészőben futó, interaktív prototípus. A felület három minta-páciens és három minta-kezelőt tartalmaz. A kiválasztott páciens–kezelő párhoz két egymástól független állapot adható meg: az aktív kapcsolat, valamint a kezelő eseményszerkesztési engedélye. Az állapotváltozás után a négy szerepkártya és a validációs eredmény azonnal újraszámításra kerül.

Tesztkörnyezeti elem	Megvalósítás
Felület	Reszponzív, böngészőalapú jogosultságpróba
Tesztadatok	3 minta-páciens, 3 minta-kezelő, előre definiált kapcsolatok
Vizsgált állapotok	Aktív / inaktív kapcsolat; olvasási / szerkesztési jog
Ellenőrzés	8 követelmény-alapú automatikus jogosultsági eset
Publikáció	Privát, kizárólag tulajdonosi hozzáférésű Site

A teszt végrehajtása

A tesztelő kiválaszt egy minta-páciens és egy minta-kezelőt, majd létrehozza vagy megszünteti a kapcsolatukat. Aktív kapcsolat esetén külön megadható, hogy a Practitioner csak olvashatja, vagy

szerkesztheti is a páciens eseményeit. A rendszer ezt követően szerepkörönként megjeleníti, hogy az adott erőforrás látható, szerkeszthető vagy tiltott.



24. ábra – A jogosultságpróba végrehajtási és validációs folyamata

Jogosultsági mátrix

A következő mátrix foglalja össze az elvárt működést. A Practitioner jogosultsága minden esetben a kiválasztott pácienssel fennálló aktív kapcsolattól függ; a szerkesztési jog a kapcsolaton belül külön engedély.

Erőforrás	Admin	Patient	Practitioner	Felvételi iroda
Páciens-adatlap	Látja, szerkeszti	Sajátot látja, szerkeszti	Kapcsolat esetén látja	Azonosítót látja
Egészségügyi esemény	Látja, szerkeszti	Sajátot látja, szerkeszti	Kapcsolat szerint látja; külön joggal szerkeszti	Nem látja, nem szerkeszti
Kezelő–páciens kapcsolat	Látja, szerkeszti	Saját hozzáférést kezeli	Saját kapcsolatot látja	Látja, létrehozza, bontja

Tesztesetek és eredmények

A validátor az elvárt jogosultsági mátrixot a kiválasztott kapcsolatból számított tényleges hozzáféréssel veti össze. Az alábbi nyolc fő eset mindegyike sikeresen teljesült.

ID	Szerepkör	Vizsgálat	Elvárt eredmény	Eredmény
JOG-01	Admin	Páciens-adatlap megnyitása és módosítása	Minden páciens adata látható és szerkeszthető	Megfelelt
JOG-02	Admin	Egészségügyi esemény módosítása	Minden esemény látható és szerkeszthető	Megfelelt
JOG-03	Patient	Saját és más páciens eseményeinek elérése	Saját esemény elérhető, más páciens eseménye tiltott	Megfelelt
JOG-04	Patient	Saját kezelői hozzáférés módosítása	Saját hozzáférés megtekinthető és szabályozható	Megfelelt
JOG-05	Practitioner	Esemény megnyitása kapcsolat nélkül	Páciens és eseményei nem érhetők el	Megfelelt
JOG-06	Practitioner	Aktív kapcsolatú páciens megnyitása	Csak az engedélyezett páciens adatai láthatók	Megfelelt
JOG-07	Practitioner	Esemény módosítása külön engedéllyel és anélkül	Szerkesztés csak külön engedély esetén lehetséges	Megfelelt
JOG-08	Felvételi iroda	Kapcsolat és eseménytartalom kezelése	Kapcsolat szerkeszthető, eseménytartalom tiltott	Megfelelt



25. ábra – A nyolc fő jogosultsági tesztet összesített eredménye

Értékelés és korlátok

A nyolc fő jogosultsági eset mindegyike megfelelt az előzetesen meghatározott követelményeknek. A próba egyértelműen szemlélteti, hogy a Practitioner hozzáférése nem pusztán a szerepkörből, hanem a konkrét kezelő–páciens kapcsolatból és a külön eseményszerkesztési engedélyből származik. A Felvételi iroda jogosultsági határa szintén igazolható: szervezési adatokat és kapcsolatokat kezelhet, klinikai eseménytartalmat azonban nem.

A következő fejlesztési lépésben a JOG-01–JOG-08 esetek backend-integrációs teszteké alakíthatók. Ezekben különösen fontos a 401 Unauthorized és 403 Forbidden válaszok, a más páciensazonosítójával kezdeményezett hozzáférés, a megszüntetett kapcsolat azonnali hatása, valamint a jogosultságváltozások auditnaplózása.

2.5.8. Későbbi továbbfejlesztési lehetőségek

- EESZT-integráció (szimulált vagy demonstrációs célra)
- Egészségügyi intézmények kezelése
- Több rendelő támogatása
- Elektronikus beutalók és mindenféle dokumentumok feltöltése és haladó kezelése
- Egészségpénztári számlázás
- Mobilalkalmazás (React Native vagy .NET MAUI)

3. Felhasználói dokumentáció

Az EgészségÚt / MedActivities egy egészségügyi eseménykezelő és pácienskezelő rendszer. A program célja, hogy egy helyen tegye kezelhetővé a páciensekhez kapcsolódó vizsgálatokat, kezeléseket, időpontokat, dokumentumokat és megjegyzéseket. A rendszer támogatja a páciensek és az egészségügyi dolgozók közötti információáramlást, ezáltal segíti annak követését, hogy a páciensnek milyen további vizsgálatra, kezelésre vagy ellátási lépésre van szüksége.

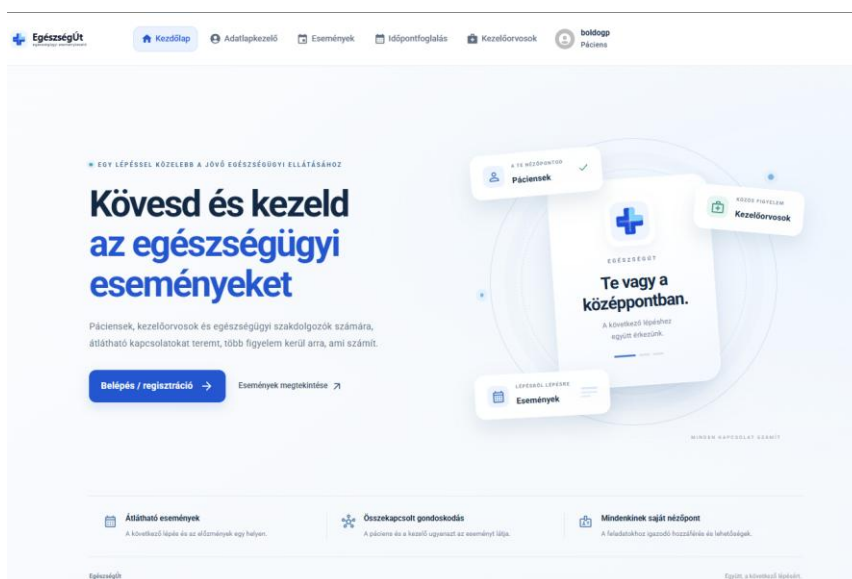
A rendszer két fő felületen használható. A páciensek számára reszponzív webalkalmazás készül, amely böngészőből, számítógépen és mobil eszközön is elérhető. Az egészségügyi dolgozók számára Windows Forms alapú asztali alkalmazás biztosítja a részletesebb adminisztrációt.

Felület	Elsődleges felhasználó	Fő cél
Webalkalmazás	Páciens	Saját adatok, események, időpontok, dokumentumok és megjegyzések megtekintése, kezelése.
Asztali alkalmazás	Orvos, asszisztens, adminisztrátor	Páciensek, egészségügyi események, időpontok és ellátási adatok adminisztrációja.

3.1. Webalkalmazás bemutatása

A webalkalmazás a páciensek és a jogosultsággal rendelkező egészségügyi szereplők böngészőből elérhető felülete. Reszponzív kialakítása miatt asztali gépen, laptopon, tableten és mobiltelefonon is használható. A felület célja, hogy az egészségügyi események, időpontok, résztvevők és megjegyzések áttekinthetően, egy helyen jelenjenek meg.

A képernyőképeken fiktív demóadatok szerepelnek. A demó szerepkörök segítségével külön nézetben próbálható ki a páciens, a kezelőorvos, a felvételi iroda és az adminisztrátor felhasználói útja.



26. ábra: A reszponzív webalkalmazás kezdőlapja

• EGY LÉPÉSEL KÖZELEBB A JÖVŐ EGÉSZSÉGÜGYI ELLÁTÁSÁHOZ

Kövesd és kezeld az egészségügyi eseményeket

Páciensek, kezelőorvosok és egészségügyi szakdolgozók számára,
átlátható kapcsolatokat teremt, több figyelem kerül arra, ami számít.

Belépés / regisztráció →

Események megtekintése ↗

Próbáld ki a saját nézőpontodból **DEMÓ**



ADMIN

Teljes áttekintés



Felvételi iroda

Szervezés és koordináció



Kezelőorvos

A kezelőorvos nézete

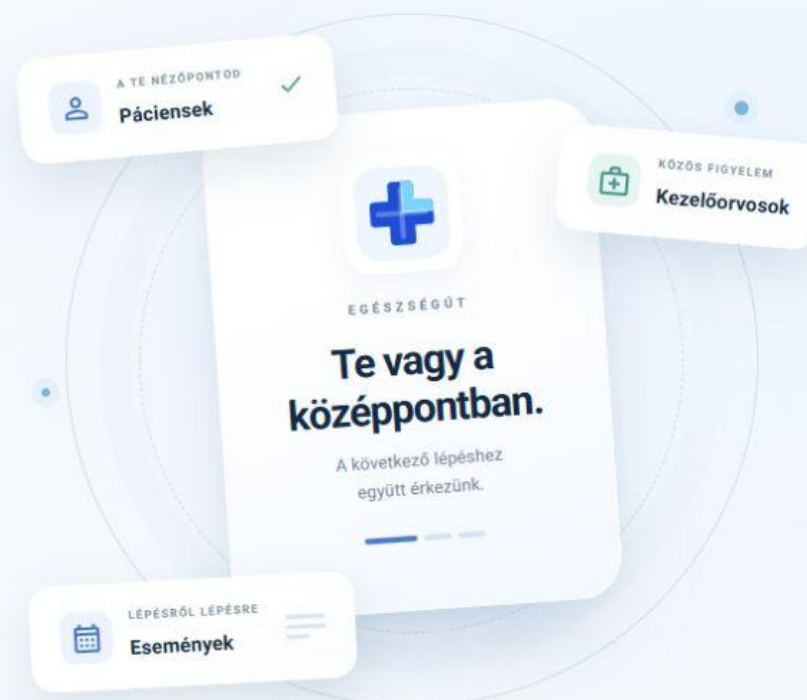


Páciens

A saját egészségutad



Négy szerepkör. Saját tesztfiókok. Szabadon kipróbálható.



MINDEN KAPCSOLAT SZÁMÍT

27. ábra: A vizsgaremek védére készült demonstrációs webalkalmazás kezdőlapja és demó szerepkörválasztó responzív felülete

A webalkalmazás főbb funkciói:

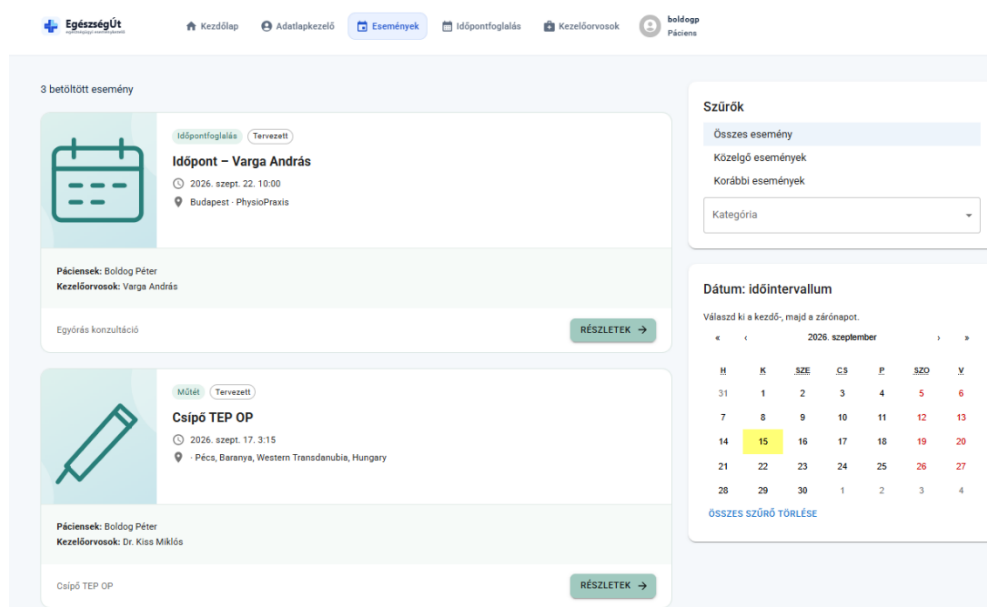
- belépés vagy regisztráció után saját egészségügyi események megtekintése,
- közelgő és korábbi események listázása, szűrése és naptár szerinti áttekintése,
- eseményrészletek megnyitása, ahol látható a helyszín, időpont, kategória, páciens és kezelőorvos,
- megjegyzések és kommunikációs felület használata az eseményhez kapcsolódó információk követésére,
- szerepkörhöz kötött hozzáférés adminisztrációs oldalakhoz, például páciens- és kezelőorvos-kezeléshez.

A webalkalmazás használata során a páciens számára fontos, hogy mindig pontos adatokat adjon meg. A rendszer űrlapvalidációval segíti a hibás vagy hiányos adatok kiszűrését, például kötelező mezők, dátumok vagy helyszínadatok esetén.

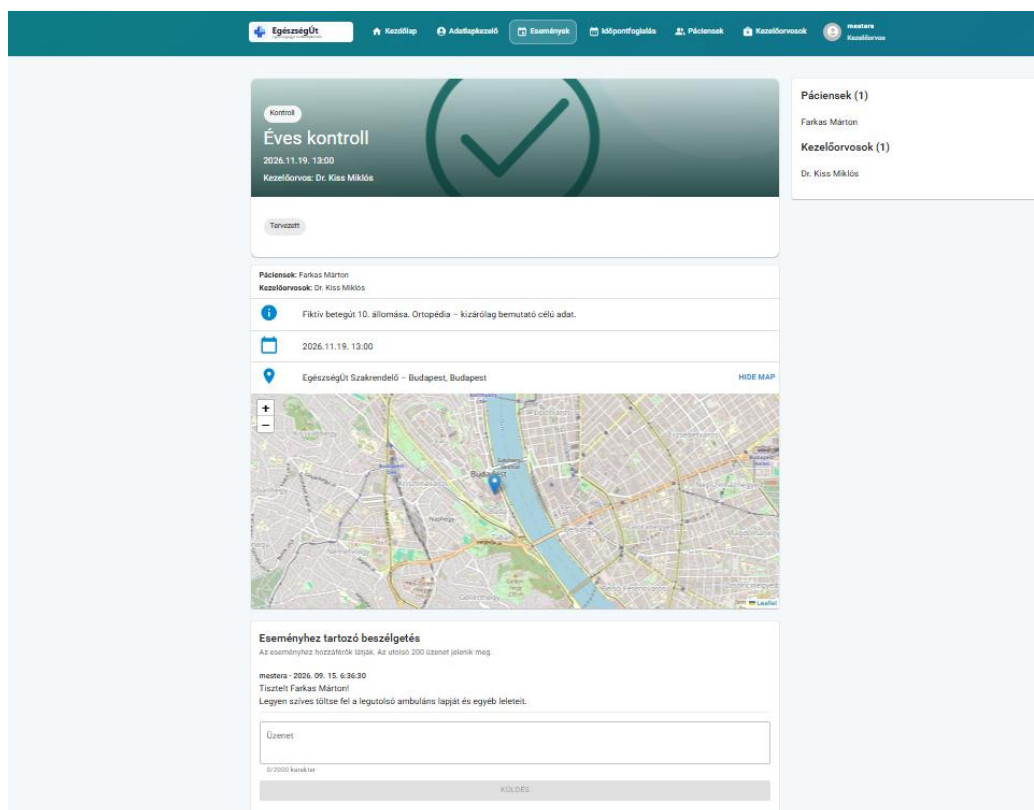
Alapvető használati lépések:

Lépés	Művelet	Eredmény
1.	A felhasználó megnyitja az EgységÚt webalkalmazást	Megjelenik a kezdőfelület.
2.	Regisztrál vagy bejelentkezik.	A rendszer azonosítja a felhasználót.
3.	Sikeres bejelentkezés után megnyitja az Események felületet.	Láthatóvá válnak a hozzá kapcsolódó események.
4.	A lista és a naptár segítségével kiválasztja a megfelelő vizsgálatot, kezelést vagy konzultációt. A Megtekintés gombbal megnyitja az esemény részleteit.	Megjelennek a részletes adatok, megjegyzések és csatolmányok.
5.	A részletező nézetben ellenőrzi az időpontot, a résztvevőket, a helyszínt és az eseményhez tartozó megjegyzéseket.	Az információ az adott eseményhez kapcsolódik.

Az eseménylista nézetben a felhasználó gyorsan áttekintheti, hogy hány esemény érhető el számára, milyen időpontban történik a következő vizsgálat vagy kezelés, és kik kapcsolódnak az adott betegüteseményhez.



28. ábra: Webes eseménylista szűrőkkel és naptárnézettel



29. ábra: Webes eseményrészletező nézet résztvevőkkel és megjegyzésfelülettel

A részletező nézet a kiválasztott esemény teljesebb adatlapja. Innen érhető el az esemény szerkesztése, a résztvevői lista, a helyszínadat, valamint az eseményhez kapcsolódó kommunikációs terület.

3.2. Az asztali alkalmazás bemutatása

Az asztali alkalmazás a Pácienskezelő Windows Forms kliens, amely az egészségügyi dolgozók, például orvosok, asszisztensek és adminisztrátorok munkáját támogatja. A Windows Forms alapú program célja, hogy gyors, áttekinthető és hatékony adminisztrációs felületet biztosítson a napi ellátási feladatokhoz.

Az alkalmazásban a Felvételi iroda vagy az egészségügyi dolgozó pácienseket kereshet, új pácienszt rögzíthet, módosíthatja a meglévő adatokat, valamint egészségügyi eseményeket, vizsgálatokat és kezeléseket kapcsolhat a pácienshez. A cél az, hogy az ellátási folyamat adminisztratív része kevesebb papírmunkával és kevesebb adatvesztési lehetőséggel történjen.

Az asztali alkalmazás főbb funkciói:

- Páciensek listázása, keresése név, TAJ-szám, e-mail, telefon vagy lakcím alapján,
- Új páciens rögzítése és meglévő páciensadatok módosítása és törlése,
- Vizsgálatok, kezelések és egyéb egészségügyi események adminisztrációja,
- TAJ-szám, születési dátum, e-mail és kötelező mezők ellenőrzése mentés előtt,
- Pácienshez kapcsolt egészségügyi események megjelenítése, létrehozása és módosítása,
- Kezelőorvos hozzárendelése eseményhez, valamint eseménystátusz kezelése,
- foglalási események védelme: ezek a webes időpontkezelőben módosíthatók, hogy az időpontütközések kezelése egységes maradjon.

EgészségÚt – Pácienskezelő

EgészségÚt / Pácienskezelő

API címe

https://localhost:5001/api

Felhasználó

admin@example.undefined

Jelszó

Bejelentkezés

Kijelentkezés

admin@example.undefined · Admin · közös SQL Server API

Név vagy TAJ keresése

Frissítés

Webes adminisztráció

Név	TAJ	Születés	Email	Telefon
Balogh Zsófia	810010107	1980. 11. 11.	paciens11@example.invalid	
Boldog Péter	003545333	2026. 09. 15.	boldogp@email.com	
Farkas Márton	810010097	1978. 10. 10.	paciens10@example.invalid	
Fekete István	810010190	1998. 08. 20.	paciens20@example.invalid	
Horváth Attila	001002004	2000. 01. 01.	horvatha@email.com	
Horváth Júlia	810010042	1968. 05. 05.	paciens5@example.invalid	
Juhász Bence	810010138	1986. 02. 14.	paciens14@example.invalid	

Új páciens

Szerkesztés

Törölés

Adatlap betöltése

Dokumentumok / megjegyzések

Boldog Péter · TAJ: 003545333

Egészségügyi események

Időpontok

Cím	Dátum	Állapot	Helyszín
Időpont – Varga András	2026. 09. 22. 10:00	Scheduled	PhysioPraxis
Csipő TEP OP	2026. 09. 17. 3:15	Scheduled	Pécs, Baranya, Western Transdanubia, Hungary
Időpont – Dr. Horváth Orsolya	2026. 09. 15. 13:00	Scheduled	EgészségÚt Szakrendelő – Debrecen

Új esemény

Szerkesztés

Törölés

30. ábra: A Pácienskezelő Windows Forms alkalmazás főablaka

Alapvető használati lépések:

Lépés	Művelet	Eredmény
1.	Az egészségügyi dolgozó elindítja a Pácienskezelő Windows Forms alkalmazást.	Megjelenik a bejelentkezési felület. Az alkalmazás automatikusan csatlakozik az alapértelmezett adatbázishoz.
2.	A felhasználó a keresőmezővel kiválasztja a megfelelő páciens.	Megjelennek a páciens alapadatai és előzményei. A pácienshez kapcsolódó események a jobb oldali listában jelennek meg.
3.	Új eseményt, vizsgálatot vagy kezelést rögzít.	Az adat az adott pácienshez kapcsolódik.
4.	Új páciens vagy új esemény rögzítésekor a program űrlapon kéri be az adatokat, majd validáció után menti azokat.	Az információ később visszakereshetővé válik.

The screenshot displays the 'EgészségÚt / Pácienskezelő' application. The main window shows a list of patients with columns for Name and TAJ (National Identification Number). The 'Páciens szerkesztése' (Edit Patient) form is overlaid on the main window. The form contains the following fields:

- Név * (Name): Papp László
- TAJ (9 számjegy) * (TAJ (9 digits)): 810010114
- Születési dátum * (Date of Birth): 1982. 12. 12.
- E-mail: paciens12@gmail.com
- Telefon (Telephone):
- Lakcím (Address): Debrecen, Kossuth Lajos utca 14.
- Általános megjegyzés (General Notes):

The form also includes 'Mentés' (Save) and 'Mégse' (Cancel) buttons. The background application window shows a list of patients with columns for Name and TAJ. The patient 'Papp László' is highlighted in the list.

31. ábra: Páciensadatok módosítása a Windows Forms alkalmazásban

A páciensűrlapon kötelező a név, a TAJ-szám és a születési dátum megadása. A megjegyzés mező belső adminisztrációs információk rögzítésére használható.

EgészségÚt – Pácienskezelő

EgészségÚt / Pácienskezelő

API címe: https://localhost:5001/api
Felhasználó: admin@example.undefined
Jelszó:

Bejelentkezés

Név vagy TAJ keresése

Név
Lakatos Emese
Mészáros András
Molnár Ádám
Nagy Anna
Németh Katalin
Oláh Noémi
Papp László

Új páciens

Papp László - TAJ: 8

Egészségügyi események

Cím
Kezelés
Szakorvosi kontroll
Szakorvosi kontroll
Szakorvosi kontroll
Szakorvosi kontroll
Szakorvosi kontroll
Diagnosztikai vizsgálat

Új esemény

Szerkesztés

Törölés

Egészségügyi esemény

Esemény címe *

Szakorvosi kontroll

Dátum és idő *

2026.08.08. 15:00

Páciens *

Papp László

Kategória *

control

Város

Debrecen

Helyszín *

EgészségÚt Szakrendelő – Debrecen

Állapot

Completed

Leírás *

Fiktív betegút 5. állomása. Gyógytorna – kizárólag bemutató célú adat.

Kezelőorvosok *

☐ Dr. Balogh Ildikó
☐ Dr. Farkas Levente
☐ Dr. Horváth Orsolya
☐ Dr. Juhász Sándor
☐ Dr. Kiss Miklós
☐ Dr. Kovács Ágnes
☐ Dr. Molnár Zoltán
☒ Dr. Nagy Balázs
☐ Dr. Németh Gabriella
☐ Dr. Papp Gergely
☐ Dr. Rácz Andrea
☐ Dr. Szabó Judit
☐ Dr. Takács Krisztina
☐ Dr. Tóth András
☐ Dr. Varga Éva
☐ Mester Ádám
☐ Varga András

Mentés

Mégse

Szakorvosi kontroll	2026. 08. 05. 11:00	Completed	EgészségÚt Szakrendelő – Szeged
Diagnosztikai vizsgálat	2026. 07. 20. 11:00	Completed	EgészségÚt Szakrendelő – Debrecen

32. ábra: Egészségügyi esemény módosítása a Windows Forms alkalmazásban

Az eseményúrlapon megadható a cím, a dátum, a leírás, a kategória, az állapot, a város, a helyszín és a kezelőorvos. A mentés előtt a kliens ellenőrzi a kötelező mezőket és a kapcsolatok érvényességét.

4. Összegzés

Az EgészségÚt vagy MedActivities projekt egy komplex, többplatformos egészségügyi eseménykezelő és pácienskezelő rendszer. A projekt célja, hogy a páciensek, orvosok és egészségügyi dolgozók egy központi rendszeren keresztül tudják kezelni az egészségügyi vizsgálatokhoz, kezelésekhöz és betegüteseményekhez kapcsolódó adatokat, dokumentumokat és megjegyzéseket. A projekt egy valós problémára nyújt megoldást, hogy egy közös alkalmazásban legyen kezelhetőek maguk az egészségügyi események, amelyeken keresztül egy kommunikációs útvonalat is kiépítünk a kezelőorvos, az egészségügyi szakdolgozó /„Practitioner”/ és a páciensek között. Ezáltal a webes applikáció segítséget nyújt az ellátási folyamat átláthatóbbá tételében. Lehetővé teszi, hogy az érintett szereplők nyomon kövessék, hogy egy páciensnek milyen vizsgálatra, kezelésre vagy további ellátási lépésre van szüksége.

Ezek az alkalmazások integrálhatóak a meglévő egészségügyi rendszerekbe, továbbá a következő funkciók használatára is alkalmas: a páciensek nyilvántartása, az egészségügyi események rögzítése és kezelése, a páciens és kezelő szakember kapcsolata, az időpontfoglalás, valamint a leletek feltöltése, rendszerezése is beépíthető továbbfejlesztési modul.

A backend ASP.NET Core REST API, az alkalmazási logika MediatR/CQRS mintát követ, az adatelérés Entity Framework Core-on keresztül történik, az MVP adatbázisa SQLite. A webes kliens React, az asztali kliens C# Windows Forms. A végleges architektúrában mindkét kliens ugyanazt az API-t használja.

A projekt kiindulópontja egy közismert social-media felület Activity-alapú alkalmazás részegysége, aminek megfelelő a vizuális felülete a széleskörű egyszerű használatra. A projekt fejlesztése során egy Activity-alapú oktatóprojekt szerkezeti mintái is felhasználásra kerültek, például a Activity CRUD műveletei, amik alapján készítettük el a saját kódjainkat. Az Activity CRUD, MediatR, AutoMapper, EF Core és SQLite alapokra kerül rá a Patient, Practitioner, Patient–Activity kapcsolat, időpontfoglalás, Identity és validáció.

A vizsgáremekünk bemutatja a backend API-fejlesztés, a webes frontend, az asztali alkalmazásfejlesztés, az adatbázisstervezés, a dokumentumkezelés, a jogosultságkezelés és a tesztelés gyakorlati alkalmazását.

A fejlesztés során törekedtünk arra, hogy a rendszer valós problémára adjon választ: segítse a páciensek és egészségügyi dolgozók közötti kommunikációt, valamint átláthatóbbá tegye az ellátási folyamatot és a betegút követését.

5. Ábrajegyzék

1. ábra: Az EgészségÚt / MedActivities adatbázisának egyszerűsített ER diagramja
2. ábra: A webalkalmazás fő felhasználói folyamatának drótváza
3. ábra: Páciens kezdőfelület vázlata
4. ábra: Egészségügyi esemény részletező felület vázlata
5. ábra: Egészségügyi dolgozói adminisztrációs felület vázlata
6. ábra: REST API végpontok vezérlése kódrészlete
7. ábra: Események listázása CQRS Queryvel kódrészlete
8. ábra: Új esemény rögzítése Commanddal kódrészlete
9. ábra: Esemény módosítása több módszerrel kódrészlete
10. ábra: Adatbázis és szolgáltatások regisztrálása kódrészlet
11. ábra: Központi API kliens és hibakezelés kódrészlete
12. ábra: Adatlekérés és cache React Queryvel kódrészlete
13. ábra: Űrlapvalidáció Zoddal kódrészlete
14. ábra: Esemény létrehozása és módosítása egy űrlapon kódrészlete
15. ábra: Részletező nézet és megjegyzésfelület kódrészlete
16. ábra: A MediatR közvetítő szerepe a frontend és az adatbázis között
17. ábra: CreateActivity.Command továbbítása MediatR segítségével
18. ábra: Páciensadatok validálása és mentése kódrészlete
19. ábra: Kiválasztott páciens Activity rekordjainak betöltése kódrészlete
20. ábra: Activity adatok validálása és mentése kódrészlete
21. ábra: Pácienskeresés SQL LIKE feltétellel kódrészlete
22. ábra: Activity rögzítése adatbázisba kódrészlete
23. ábra: Jogosultsági modell
24. ábra: Tesztelési folyamat
25. ábra: Teszteredmények
26. ábra: A reszponzív webalkalmazás kezdőlapja
27. ábra: A vizsgaremek védére készült demonstrációs webalkalmazás kezdőlapja és demó szerepkörválasztó reszponzív felülete
28. ábra: Webes eseménylista szűrőkkel és naptárnézettel
29. ábra: Webes eseményrészletező nézet résztvevőkkel és megjegyzésfelülettel
30. ábra: A Pácienskezelő Windows Forms alkalmazás főablaka
31. ábra: Páciensadatok módosítása a Windows Forms alkalmazásban
32. ábra: Egészségügyi esemény módosítása a Windows Forms alkalmazásban

6. Forrásjegyzék

ASP.NET documentation (<https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0>)

MICROSOFT.COM - ASP.NET Core (<https://dotnet.microsoft.com/en-us/apps/aspnet>)

ASP.NET overview - LEARN.MICROSOFT.COM (<https://learn.microsoft.com/en-us/aspnet/overview>)

SignalR – LEARN.MICROSOFT.COM (<https://learn.microsoft.com/en-us/aspnet/signalr/>)

ASP.NET Identity – LEARN.MICROSOFT.COM (<https://learn.microsoft.com/en-us/aspnet/identity/>)

Entity Framework– LEARN.MICROSOFT.COM (<https://learn.microsoft.com/en-us/aspnet/entity-framework>)

HTML, CSS, JavaScript (<https://developer.mozilla.org/en-US/>)

React (<https://react.dev/>)

Material UI (<https://mui.com/material-ui/getting-started/>)

Complete guide to building an app with .Net Core and React (<https://www.udemy.com/course/complete-guide-to-building-an-app-with-net-core-and-react/>)